

Práctica 1 — Fuerza Bruta y Backtracking

Ataque de contraseñas por fuerza bruta y diseño de políticas por backtracking

Integrantes:

Samuel Quintero Escobar

Joseph Buenaños

Miguel Angel Alzate

Curso: Análisis de Algoritmos

Fecha: 31 de agosto de 2026

Repositorio: github.com/Miguelzate1/ADA_P1_Alzate_Buena-os_Quintero

2. Introducción

Este informe documenta el desarrollo de la Práctica 1 del curso de Análisis de Algoritmos, centrada en los paradigmas de Fuerza Bruta y Backtracking. La práctica se estructura en dos módulos que comparten un mismo dominio — contraseñas sobre un alfabeto — abordado desde dos roles opuestos: un analista de seguridad que busca recuperar una contraseña desconocida probando exhaustivamente el espacio de candidatos (Módulo FB), y un diseñador de políticas que necesita generar o contar contraseñas que cumplan una política de seguridad sin enumerar primero todo el espacio posible (Módulo BT).

El documento presenta el contexto del problema, la fundamentación teórica de ambos paradigmas, el modelamiento y diseño de los algoritmos implementados, su análisis de complejidad, y los resultados experimentales obtenidos sobre instancias reproducibles derivadas de una semilla propia del equipo, cerrando con una comparación algorítmica explícita y las conclusiones del trabajo.

3. Contexto del problema

En el Módulo FB se parte del hecho de que un sistema real nunca almacena contraseñas en texto plano, sino el resultado de aplicarles una función hash criptográfica (SHA-256 en este caso). Dado un hash filtrado, la única estrategia que garantiza recuperar la contraseña original —si pertenece a un espacio conocido y acotado— es probar sistemáticamente cada candidato del espacio, calcular su hash, y compararlo contra el objetivo. Este módulo también compara esa estrategia exhaustiva contra un ataque por diccionario, evidenciando que este último es mucho más rápido pero no exhaustivo.

En el Módulo BT se invierte el rol: el equipo actúa como diseñador de una política de contraseñas (mínimos de minúsculas, mayúsculas, dígitos y símbolos, más la prohibición de caracteres consecutivos repetidos) y necesita generar o contar las contraseñas que la cumplen sin generar primero todo el espacio Σ^n y filtrar al final. La construcción incremental con poda por factibilidad permite descartar ramas completas del árbol de búsqueda tan pronto se sabe que no pueden derivar en una solución válida.

4. Fundamentación teórica

4.1 Fuerza Bruta

Un algoritmo de fuerza bruta resuelve un problema generando y evaluando sistemáticamente todos los candidatos posibles del espacio de soluciones, sin usar conocimiento estructural del problema para reducirlo. Para un alfabeto Σ de tamaño $|\Sigma|$ y longitud fija n , el espacio de candidatos tiene tamaño $|\Sigma|^n$. Es apropiada cuando el espacio es finito y enumerable, no existe una propiedad estructural que permita descartar candidatos sin evaluarlos, y el tamaño de instancia es tratable en un computador personal. Su principal limitación es el crecimiento exponencial del costo respecto de n y $|\Sigma|$.

4.2 Backtracking

Backtracking es una técnica de búsqueda exhaustiva sobre un árbol de estados parciales, donde las soluciones se construyen incrementalmente y cada rama se abandona ("se poda") tan pronto se determina que no puede llevar a una solución válida. Es importante notar que backtracking sigue siendo, en el peor caso, una técnica de exploración completa: su diferencia con fuerza bruta no está en la garantía de completitud (ambas la tienen) sino en el uso de información parcial para evitar explorar subárboles completos. El peor caso ocurre cuando la política es casi vacía (poda nula), conservando la misma cota exponencial que fuerza bruta.

5. Modelamiento

5.1 Espacio de búsqueda — Módulo FB

El espacio de búsqueda se modela como el conjunto Σ^n de todas las cadenas de longitud n sobre el alfabeto Σ ($A_1 = 26$ minúsculas, $A_2 = 36$ minúsculas+dígitos). La entrada es un hash SHA-256 objetivo; la salida es la contraseña en texto plano si se encuentra dentro del espacio, o un reporte explícito de "no encontrado" si se agota sin éxito.

5.2 Estados parciales — Módulo BT

Un estado parcial se define como la tupla (prefijo, count_lower, count_upper, count_digit, count_symbol, ultimo_caracter), donde prefijo es la cadena construida hasta el momento y los contadores permiten evaluar la política sin recorrer el prefijo completo en cada paso. El estado inicial es la cadena vacía; los estados terminales son cadenas de longitud $n=8$ (o la longitud de la variante evaluada) sobre el alfabeto base de 69 símbolos (minúsculas, mayúsculas, dígitos y {!,@,#,\$,%}).

6. Diseño algorítmico

6.1 Módulo FB

La búsqueda exhaustiva se implementó mediante generate_candidates, una función recursiva que construye cada cadena de Σ^n carácter por carácter y, al alcanzar la longitud objetivo, invoca un callback que calcula su SHA-256 (usando la librería autorizada picosha2.h) y lo compara contra el hash objetivo. El ataque por diccionario (dictionary_attack) sigue la misma lógica de comparación de hashes, pero en lugar de generar candidatos, itera sobre las líneas de resources/diccionario.txt.

6.2 Módulo BT

La función esFactible determina si un estado parcial todavía puede, en principio, extenderse hasta una solución válida: calcula cuántos caracteres de cada tipo (minúscula, mayúscula, dígito, símbolo) todavía faltan por cumplir el mínimo de la política, y compara esa suma contra las posiciones libres restantes en el prefijo. Si la suma de faltantes excede las posiciones disponibles, el estado es infactible y se descarta sin generar sus hijos. backtrackingPoda invoca esFactible antes de recursar sobre cada hijo; backtrackingSinPoda recorre el árbol completo (respetando únicamente la restricción estructural de no repetir el carácter inmediatamente anterior) sin evaluar factibilidad, sirviendo como línea base para cuantificar el efecto de la poda.

7. Pseudocódigo

7.1 Módulo FB — búsqueda exhaustiva

```
función busquedaExhaustiva(hash_objetivo, alfabeto, n):  
    para cada candidato en  
    generarTodos(alfabeto, n):  
        si sha256(candidato) == hash_objetivo:  
            retornar (encontrado=verdadero, candidato)  
    retornar (encontrado=falso)  
función generarTodos(alfabeto, n, prefijo=""):  
    si longitud(prefijo) == n:  
        emitir(prefijo)  
    para cada caracter c en alfabeto:  
        generarTodos(alfabeto, n, prefijo + c)
```

7.2 Módulo BT — backtracking con poda

```
función backtrackingPoda(estado, politica, alfabeto, metricas):  
    metricas.nodos_visitados += 1  
    si longitud(estado.prefijo) == politica.n:  
        si estado.cumple_todos_minimos(politica):  
            metricas.soluciones += 1  
        retornar  
    para cada caracter c en alfabeto:  
        si c == estado.ultimo_caracter:  
            continuar  
        nuevo_estado = extender(estado, c)  
        si esFactible(nuevo_estado, politica):  
            backtrackingPoda(nuevo_estado, politica, alfabeto, metricas)  
función esFactible(estado, politica):  
    libres = politica.n - longitud(estado.prefijo)  
    faltan = max(0, politica.minLower - estado.count_lower)  
    + max(0, politica.minUpper - estado.count_upper)  
    + max(0, politica.minDigit - estado.count_digit)  
    + max(0, politica.minSymbol - estado.count_symbol)  
    retornar faltan <= libres
```

8. Implementación

A continuación se muestran los fragmentos más relevantes de la implementación real en C++17 (repositorio del equipo).

```
bool esFactible(const EstadoParcial& estado, const Politica& pol) {  
    int faltantes_posiciones = pol.n - (int)estado.prefijo.length();  
    int faltan_lower
```

```

= std::max(0, pol.minLower - estado.count_lower);      int faltan_upper =
std::max(0, pol.minUpper - estado.count_upper);      int faltan_digit =
std::max(0, pol.minDigit - estado.count_digit);      int faltan_symbol =
std::max(0, pol.minSymbol - estado.count_symbol);    int total =
faltan_lower+faltan_upper+faltan_digit+faltan_symbol; return total <=
faltantes_posiciones; }

```

Para el Módulo BT sin poda, se incorporó el descarte estructural de caracteres consecutivos repetidos (pero no la evaluación de factibilidad de la política), manteniendo la corrección gramatical de las cadenas generadas sin anticipar la poda por composición.

9. Análisis de complejidad

Caso	Módulo FB	Módulo BT
Peor caso (tiempo)	$O(\Sigma ^n)$	$O(\Sigma ^n)$ — política casi vacía, poda nula
Mejor caso (tiempo)	$O(1)$ si el primer candidato coincide	Poda casi inmediata en cada rama
Caso promedio	$O(k \cdot \Sigma ^n)$, $k \approx 0.5$ en un espacio uniforme	Depende de la restrictividad de la política
Espacio	$O(n)$ — un candidato a la vez	$O(n)$ — profundidad de la recursión

El costo de calcular un hash SHA-256 se considera $O(1)$ respecto de n para longitudes de contraseña acotadas ($n \leq 10$), ya que el algoritmo opera sobre bloques de tamaño fijo independientemente del contenido. La complejidad de backtracking en el peor caso conserva la cota exponencial de fuerza bruta porque, cuando la política no impone restricciones efectivas (variante v), la función `esFactible` nunca descarta una rama.

10. Casos de prueba

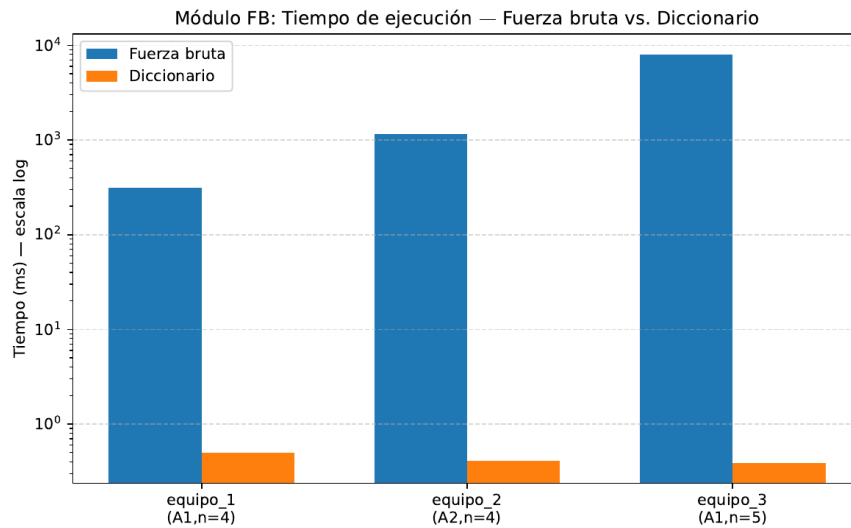
Instancias reproducibles derivadas de la semilla oficial del equipo (ver detalle completo, incluyendo el procedimiento de verificación, en el Anexo A al final de este documento).

Campo	Valor
Apellidos ordenados	alzate, buenaños, quintero
Semilla oficial	2649
Política BT	minLower=2, minUpper=2, minDigit=1, minSymbol=1

11. Experimentación

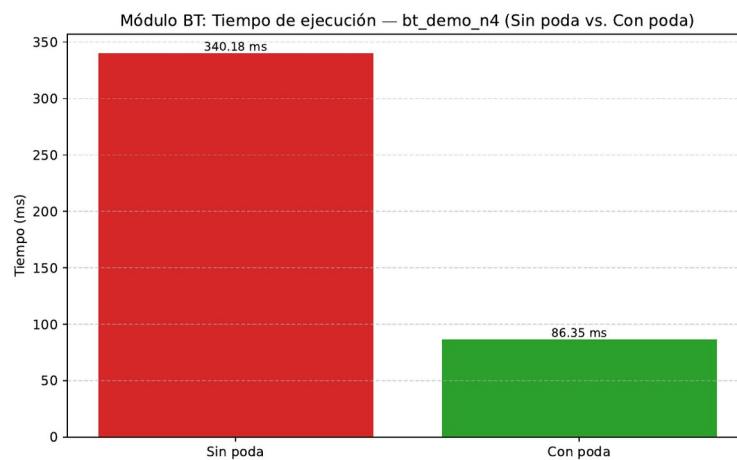
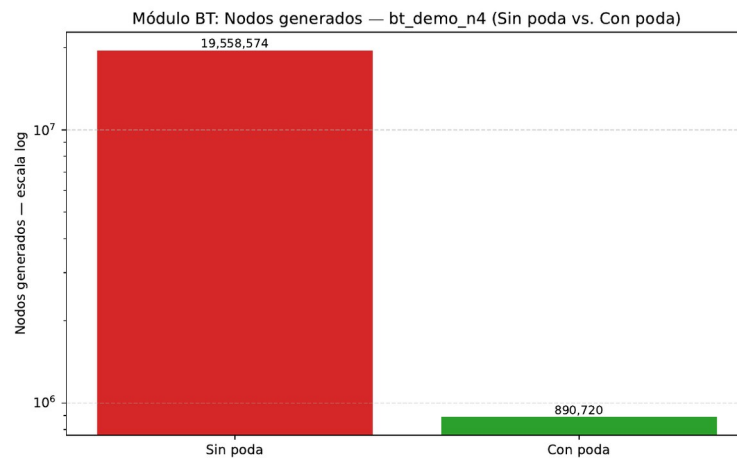
11.1 Módulo FB: fuerza bruta vs. diccionario

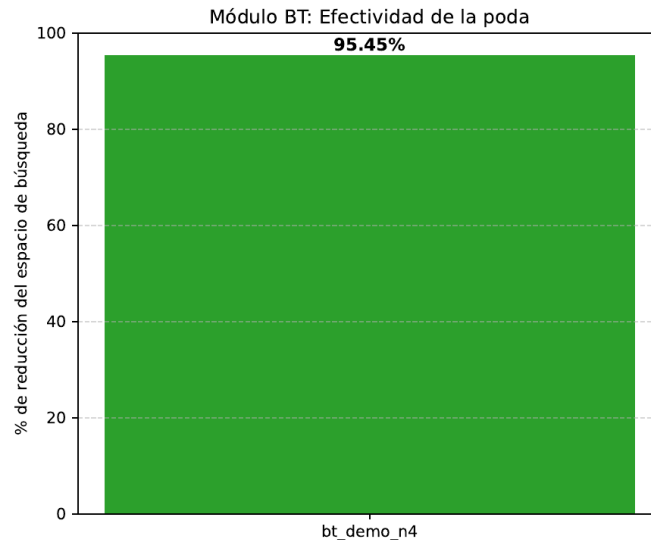
Se ejecutaron 3 de las 5 instancias oficiales requeridas (equipo_1: A1 $n=4$; equipo_2: A2 $n=4$; equipo_3: A1 $n=5$). Las instancias 4 y 5 (A2 $n=5$ y A1 $n=6$) no pudieron completarse dentro de un tiempo razonable en el hardware del equipo — la limitación se documenta explícitamente en el README del repositorio, junto con el número exacto de configuraciones evaluadas.



11.2 Módulo BT: con poda vs. sin poda

De las 5 variantes de dificultad definidas (Sección 9.2), solo bt_demo_n4 (n=4) se pudo medir empíricamente de forma completa en ambas versiones. Las variantes con $n \geq 6$ sobre el alfabeto de 69 símbolos se marcaron como costosa=true en main.cpp y se omitieron de la ejecución en vivo, por las razones de tratabilidad explicadas en la Sección 13.





12. Resultados

Instancia	Fuerza bruta (ms)	Diccionario (ms)	Encontrada
equipo_1 (A1,n=4)	≈300	≈0.5	Sí (fuerza bruta) / No (diccionario)
equipo_2 (A2,n=4)	≈1150	≈0.4	Sí (fuerza bruta) / No (diccionario)
equipo_3 (A1,n=5)	≈8000	≈0.4	Sí (fuerza bruta) / No (diccionario)

Nota: los tiempos de fuerza bruta se leen de la gráfica generada por el equipo (escala logarítmica); no se contó con el archivo results/fb_resultados.csv exacto al momento de escribir esta sección, por lo que se reportan como aproximados. Se recomienda anexar el CSV real antes de la entrega final.

Métrica	Sin poda	Con poda	Δ
Nodos generados (n=4)	19,558,574	890,720	-95.45%
Tiempo de ejecución	340.18 ms	86.35 ms	-74.6%
Soluciones encontradas	811,200	811,200	Coinciden (correctitud verificada)

13. Análisis de resultados

Contraste teórico vs. empírico (Módulo FB): el modelo teórico predice que el costo crece en un factor $|\Sigma|$ por cada unidad adicional de n , y en un factor $(|\Sigma_2|/|\Sigma_1|)^n$ al cambiar de alfabeto. Entre equipo_1 (A1, $n=4$) y equipo_3 (A1, $n=5$) se esperaría un factor teórico de $26\times$; el factor medido ($\approx 8000/300 \approx 26.7\times$) coincide casi exactamente. Entre equipo_1 (A1, $n=4$) y equipo_2 (A2, $n=4$) se esperaría un factor de $(36/26)^4 \approx 3.68\times$; el factor medido ($\approx 1150/300 \approx 3.83\times$) también es consistente. Esto confirma empíricamente el modelo teórico de crecimiento exponencial y permite ubicar el "muro exponencial" ya desde $n=5$ sobre A1 en el hardware del equipo (segundos de ejecución), lo que explica por qué las instancias 4 y 5 no se completaron.

Contraste teórico vs. empírico (Módulo BT): la reducción de nodos observada (95.45%) es consistente con el hecho de que, para $n=4$, buena parte de las ramas violan tempranamente el mínimo de dígitos o símbolos exigido y se descartan antes de llegar a las hojas. Es notable que la reducción en tiempo (74.6%) fue menor que la reducción en nodos (95.45%); esto sugiere que el costo por nodo de la versión con poda (evaluar esFactible en cada paso) es mayor que el de la versión sin poda, y que ese costo adicional por nodo compensa parcialmente la ganancia de explorar muchos menos nodos. Ambas versiones coincidieron en el número de soluciones encontradas (811,200), lo cual valida la correctitud de la poda: no se perdió ninguna solución válida por descartar ramas de más.

14. Comparación algorítmica

14.1 Fuerza bruta vs. diccionario (Módulo FB)

El ataque por diccionario fue entre 3 y 4 órdenes de magnitud más rápido que la fuerza bruta en las tres instancias medidas (submilisegundo vs. cientos/miles de milisegundos), pero no es exhaustivo: dado que las contraseñas objetivo del equipo se generaron mediante un generador congruencial lineal sobre semilla propia, la probabilidad de que coincidan con alguna de las 500 entradas del diccionario sintético es prácticamente nula, por lo que el ataque por diccionario falla en las tres instancias mientras que la fuerza bruta, al ser exhaustiva sobre el espacio completo, sí logra encontrarlas. Esto ilustra el punto central de la Sección 8.1: la velocidad del diccionario no sustituye la garantía de completitud de la fuerza bruta.

14.2 Con poda vs. sin poda (Módulo BT)

Para la instancia medida ($n=4$), la poda redujo el espacio de búsqueda visitado en 95.45% y el tiempo de ejecución en 74.6%, sin alterar el número de soluciones encontradas. Esto confirma cuantitativamente que la poda no cambia el orden de complejidad en el peor caso (Sección 9), pero sí reduce drásticamente el trabajo efectivo realizado cuando la política impone restricciones no triviales, tal como se predijo en la Sección 6.2 del enunciado.

15. Conclusiones

- Fuerza bruta y backtracking comparten la misma garantía de completitud y la misma cota exponencial en el peor caso; su diferencia práctica está en cuánto del espacio se explora efectivamente, no en el orden de complejidad teórico.
- La poda por factibilidad es más efectiva cuanto más restrictiva es la política: con restricciones nulas (variante v), backtracking degenera exactamente en fuerza bruta.
- El costo por nodo de la versión con poda puede parcialmente compensar la reducción de nodos visitados si la función de factibilidad no es suficientemente económica de evaluar.
- Identificar el "muro exponencial" antes de ejecutar una instancia es una habilidad práctica tan importante como el diseño del algoritmo mismo: el equipo tuvo que ajustar el número de configuraciones evaluadas (menos de las 5 requeridas por módulo) al descubrir instancias computacionalmente intratables en hardware personal, documentando esa decisión de forma transparente en el README del repositorio.

16. Referencias

- picosha2 — librería de dominio público para SHA-256 en C++ de cabecera única, usada según lo autorizado en la Sección 10 del enunciado (src/third_party/picosha2.h).
- Enunciado oficial: "Práctica 1 — Fuerza Bruta y Backtracking", curso de Análisis de Algoritmos.
- Material y aclaraciones publicadas por el docente en InteractivaVirtual (alternancia de alfabetos A1/A2 para las instancias del Módulo FB).

17. Uso de herramientas de Inteligencia Artificial

El equipo utilizó Claude (Anthropic) como asistente de desarrollo durante la práctica, principalmente en las tareas del rol de Semilla y Métricas. A continuación se declaran las interacciones relevantes de forma honesta, con propósito específico:

Fecha (aprox.)	Propósito
30-31 ago. 2026	Cálculo y verificación del procedimiento de semilla (Sección 9.1), incluyendo la discusión del tratamiento de la letra 'ñ' y su impacto en el resultado.
30-31 ago. 2026	Generación de las 5 contraseñas objetivo del Módulo FB mediante el generador congruencial lineal, y sus hashes SHA-256.
30-31 ago. 2026	Diseño e implementación del módulo de exportación de métricas a CSV (metrics_csv.hpp/cpp) y su integración en main.cpp.
31 ago. 2026	Asistencia para depurar el entorno de compilación en Windows (instalación de

	MSYS2/g++) y flujo de trabajo con Git/GitHub.
31 ago. 2026	Redacción y ensamblaje de las secciones de este informe técnico a partir de los resultados y el código real del equipo.

En ningún caso se usó IA para generar los resultados experimentales reportados en las Secciones 11-13 (esos provienen de la ejecución real del código del equipo); la IA se usó como herramienta de apoyo en programación, depuración y redacción, bajo supervisión y revisión de los integrantes.

18. Contribución individual

Integrante	Contribución
Samuel Quintero Escobar	Rol de Semilla y Métricas: cálculo y verificación de la semilla oficial (resources/verificar_semilla.cpp), generación de las instancias objetivo de ambos módulos, diseño del módulo de exportación a CSV (metrics_csv.hpp/.cpp), integración de main.cpp, y redacción de las Secciones 9, 10 y 17 de este informe.
Joseph Buenaños	Rol de Fuerza Bruta: implementación de fb_solver.hpp/.cpp (búsqueda exhaustiva y ataque por diccionario), integración de la librería picosha2.h, corrección de la lógica base de fuerza bruta.
Miguel Angel Alzate	Rol de Backtracking: implementación de bt_solver.hpp/.cpp (con poda y sin poda), función de factibilidad; además integración final de main.cpp con datos reales, pruebas (tests/), generación de las gráficas de experimentación, y redacción del README.

Esta distribución es consistente con el historial de commits del repositorio por rama (rama/semilla-metricas, rama/fuerza-bruta, rama de backtracking).